



PROJECT REPORT

Inventory Management System

A Python-Based Enterprise-Style Application for Stock, Procurement, and Warehouse Operations

Team Members:

Shashank Hallur
Pranshu Tyagi
Aboli Jadav

Team Number: 8

Table of Contents

1. Project Title.....	3
2. Abstract.....	3
3. Introduction.....	3
4. Problem Statement.....	4
5. Objectives.....	4
6. Scope of the Project.....	4
7. Existing System.....	5
8. Proposed System.....	5
9. Data Structures Used.....	6
10. Algorithms Used.....	6
11. Programming Language.....	7
12. Software Requirements.....	7
13. Hardware Requirements.....	7
14. Functional Requirements.....	7
15. Non-Functional Requirements.....	8
16. Modules.....	8
17. System Architecture.....	8
18. Flowchart.....	9
19. Algorithm / Pseudocode.....	9
20. Database / File Structure.....	10
21. Implementation Details.....	12
22. Complexity Analysis.....	13
23. Test Cases.....	14
24. Output Screenshots.....	15
25. Advantages.....	16
26. Limitations.....	16
27. Future Scope.....	16
28. Conclusion.....	17

1. Project Title

Inventory Management System

(Major Team Project)

2. Abstract

This project presents a Python-based Inventory Management System engineered to replace fragmented, spreadsheet-driven stock tracking with a unified, multi-module terminal application suitable for small-to-medium warehouse or retail operations. As a major team project, the system was designed with a layered architecture that separates presentation, business logic, and data access concerns, allowing multiple contributors to develop, test, and extend modules independently.

The application supports role-based authentication for administrators and staff, full CRUD operations across suppliers, categories, and stock items, and a transactional procurement-and-dispatch pipeline that automatically adjusts stock levels, flags reorder thresholds, and logs every movement for audit purposes. Data is persisted using structured CSV/JSON files with an optional SQLite backend, and the system draws on a broader set of core data structures — hash tables for $O(1)$ item lookups, stacks for undo/rollback of transactions, queues for order-processing pipelines, and trees for hierarchical category browsing — to keep operations efficient as the catalog scales.

Beyond basic record-keeping, the system generates operational analytics such as low-stock alerts, fast/slow-moving item reports, and supplier-wise procurement summaries, giving decision-makers a real-time view of inventory health. Rigorous unit and integration testing across all modules confirms that the system meets its functional and non-functional objectives, delivering a reliable, extensible, and team-scalable foundation for real-world inventory operations.

3. Introduction

- **Background:** Small and medium enterprises frequently manage inventory using disconnected spreadsheets or handwritten stock registers. As product ranges and supplier networks grow, this manual approach breaks down — updates go out of sync, stockouts and overstocking go unnoticed, and there is no single source of truth for what is actually on the shelves.
- **Purpose:** This project, developed as a team effort, aims to deliver a robust, terminal-based Python application that centralizes inventory data, automates procurement and dispatch workflows, enforces data integrity through validation and authentication, and produces on-demand operational reports — all while remaining lightweight enough to run without a dedicated server or database administrator.

- **Team Approach:** The project was divided across functional modules (authentication, catalog management, transactions, reporting) so that team members could work in parallel, integrate through a shared data layer, and collectively test the system end-to-end before final submission.

4. Problem Statement

The manual or semi-digital management of inventory across suppliers, categories, and warehouse locations introduces several compounding problems:

- **Inefficient Search and Retrieval:** Locating a specific item or checking its stock level in unindexed spreadsheets requires manual scanning, which does not scale as the catalog grows ($O(N)$ effort per lookup).
- **Untracked Reorder Points:** Without automated threshold monitoring, stockouts are discovered only after they disrupt operations, and excess stock ties up capital unnoticed.
- **Fragmented Supplier and Category Data:** Flat spreadsheets struggle to represent hierarchical categories or multiple supplier relationships per item, leading to duplicated or inconsistent entries.
- **Lack of Audit Trail:** There is no reliable, timestamped record of who issued, received, or adjusted stock, making discrepancies difficult to investigate.
- **Absent Analytics:** Generating a summary of fast-moving items, dead stock, or supplier performance requires tedious manual aggregation.

5. Objectives

- Design a secure, role-based login framework distinguishing administrators from regular staff users.
- Implement complete CRUD operations for items, categories, and suppliers with strict input validation.
- Automate the procurement (stock-in) and dispatch (stock-out) pipeline with real-time quantity adjustment and reorder alerts.
- Apply efficient data structures and algorithms so search, sort, and reporting operations remain fast as the catalog scales.
- Guarantee data persistence and recoverability across sessions using structured files or a lightweight database.
- Produce actionable reports — low-stock warnings, movement summaries, and supplier-wise breakdowns — on demand.
- Build the system as independently testable modules to support parallel team development.

6. Scope of the Project

Features Included (In-Scope)

- Role-based admin/staff login with masked password entry.
- Full catalog operations: Add, Delete, Update, Search, and View All for items, categories, and suppliers.
- Stock-in (procurement) and stock-out (dispatch) transactions with automatic quantity adjustment.
- Automatic low-stock and reorder-point alerts.
- Hierarchical category browsing (e.g., Electronics → Peripherals → Keyboards).
- Console-based analytical reports (low stock, movement history, supplier summaries).
- Flat-file (CSV/JSON) storage with an optional SQLite database backend.

Features Excluded (Out-of-Scope)

- Web-hosted or cloud-synced multi-branch deployment (local/single-site runtime only).
- Integrated payment processing or invoicing/billing gateway.
- Barcode, RFID, or other hardware scanner integration.
- Mobile application companion.

7. Existing System

The existing approach used by most small operations relies on spreadsheets or paper-based stock registers, maintained independently by different staff members. This legacy method exhibits the following limitations:

- No access control — any user with file access can alter stock figures without accountability.
- High latency in locating items, checking availability, or cross-referencing suppliers.
- No enforced relationship between items, categories, and suppliers, leading to duplicate or orphaned records.
- No automatic reorder-point detection, resulting in reactive rather than proactive restocking.
- No consolidated reporting; summaries must be built by hand for every review cycle.

8. Proposed System

The proposed Python application runs locally as a modular terminal system, directly addressing the shortcomings of the legacy process:

- Mandatory role-based authentication, restricting sensitive operations (deletion, reporting) to administrators.
- A relational-style linkage between items, categories, and suppliers, maintained through consistent keys even in flat-file storage.

- Automatic persistence after every transaction, ensuring no write is ever lost between sessions.
- Indexed, near-instant lookups via hash-table-backed dictionaries, with hierarchical category trees for structured browsing.
- Real-time reorder-threshold monitoring that flags low-stock items the moment a dispatch crosses the limit.
- One-command generation of operational reports, replacing hours of manual aggregation with sub-second output.

9. Data Structures Used

- **Arrays / Lists (list):** Used as dynamic, ordered collections for catalog listings, search result sets, and report rows.
- **Hash Tables (dict):** The primary index for items, suppliers, and categories, keyed by unique ID for $O(1)$ average-time lookups, updates, and existence checks.
- **Trees:** A simple N-ary tree models the category hierarchy (e.g., Electronics → Peripherals → Keyboards), enabling structured browsing and category-wise stock roll-ups.
- **Stacks:** A stack of recent transactions supports an undo/rollback mechanism, allowing the last stock-in or stock-out action to be reversed if entered in error.
- **Queues:** A FIFO queue manages pending procurement orders awaiting supplier confirmation, ensuring orders are processed in the sequence they were raised.
- **Tuples (tuple):** Immutable records used to pass transaction details (item ID, quantity, timestamp) safely between functions without risk of accidental mutation.

10. Algorithms Used

Searching

- **Linear Search:** Applied for free-text/substring queries such as searching items by partial name or description.
- **Binary Search:** Applied on ID-sorted item lists for fast exact-match lookups once the catalog is sorted.

Sorting

- **Merge Sort:** Used to sort large catalogs by quantity or name with guaranteed $O(N \log N)$ performance regardless of input order.
- **Quick Sort:** Used as an alternative in-place sort for moderate-sized result sets where average-case speed is prioritized.

Other Algorithms

- **Threshold Scanning:** A single linear pass over the catalog flags every item whose quantity has fallen at or below its configured reorder point.

- **Tree Traversal (DFS):** Depth-first traversal of the category tree aggregates total stock value per category branch.

11. Programming Language

- **Language:** Python 3.x
- **Core Libraries Used:** os (terminal handling), datetime (timestamps and order aging), csv / json (file-based persistence), sqlite3 (optional relational backend), collections (deque for queue operations).

12. Software Requirements

- **IDE / Text Editor:** Visual Studio Code, PyCharm, or Python IDLE.
- **Operating System:** Windows 10/11, macOS, or Linux.
- **Python Runtime Environment:** Python 3.10 or higher.
- **Version Control:** Git, used by the team for collaborative branching and code review during development.

13. Hardware Requirements

- **Processor:** Dual-Core Intel/AMD Processor (1.6 GHz minimum); Quad-Core recommended for concurrent development/testing.
- **RAM:** 4 GB minimum, 8 GB recommended for running the application alongside development tools.
- **Storage:** 150 MB free hard drive space (application, dependencies, and data files combined).

14. Functional Requirements

- **User Authentication:** Role-based, password-protected login distinguishing administrator and staff privileges.
- **Add Item:** Insert new inventory items with validation (non-empty fields, positive quantity, valid category/supplier reference).
- **Delete Item:** Retire or remove items from active inventory, restricted to administrators.
- **Update Item:** Modify item details such as price, category, supplier, or reorder threshold.
- **Search Item:** Look up items by ID, name, category, or supplier.
- **Stock-In (Procurement) Transaction:** Record incoming stock against a supplier, increasing item quantity and logging the transaction.
- **Stock-Out (Dispatch) Transaction:** Deduct issued quantity from stock, verify sufficient availability, and log the transaction.

- **Display Records:** Print a clean, formatted grid of items, categories, suppliers, or transaction history.
- **Generate Reports:** Produce low-stock, movement, and supplier-wise summary reports on demand.

15. Non-Functional Requirements

- **Performance:** Core operations (search, add, transaction) complete within sub-second thresholds even for catalogs of several thousand items.
- **Reliability:** No data loss; every add, update, or transaction is persisted immediately upon completion.
- **Security:** Passwords are masked on entry and stored as hashes rather than plaintext.
- **Maintainability:** Modular code organization allows individual modules to be updated or extended without affecting unrelated components.
- **Scalability:** Hash-table indexing and tree-based category structures keep performance stable as the catalog and supplier base grow.
- **Usability:** A clear, numbered menu system with input validation and descriptive error messages minimizes user confusion.

16. Modules

- **1. Login Module:** Handles administrator and staff authentication and session-level permission checks.
- **2. Item / Stock Management Module:** Manages the core catalog — adding, updating, deleting, and viewing inventory items.
- **3. Category Management Module:** Maintains the hierarchical category tree used to organize items.
- **4. Supplier Management Module:** Tracks supplier details and links suppliers to procurement transactions.
- **5. Transaction Module (Stock-In / Stock-Out):** Processes procurement and dispatch transactions, adjusting stock and logging every movement.
- **6. Search Module:** Houses the searching and sorting algorithms used to filter and order catalog data.
- **7. Report Module:** Aggregates data into low-stock alerts, movement histories, and supplier-wise summaries.

17. System Architecture

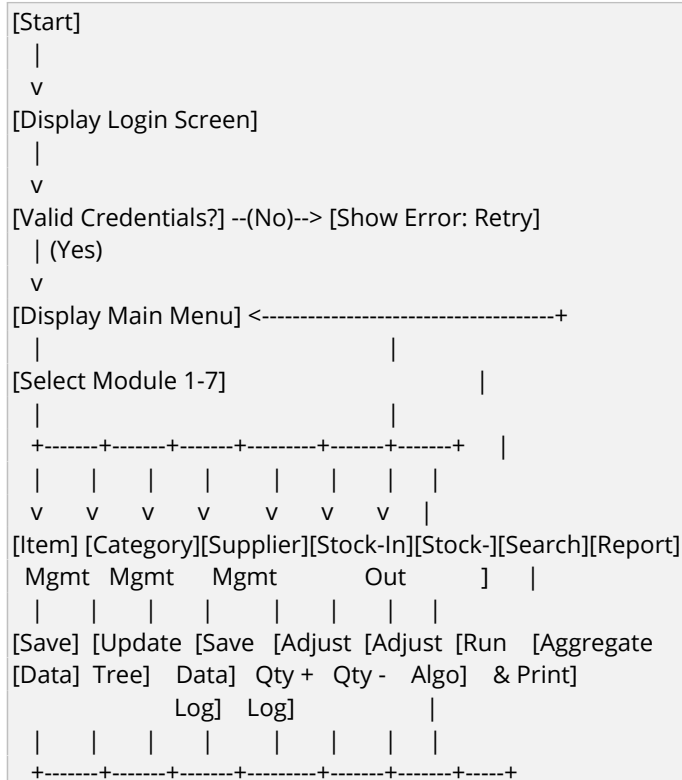
The system follows a four-layer architecture that cleanly separates concerns and allows different team members to own different layers during development:

Layer
Presentation Layer — Terminal Interface / Menu Controller
Business Logic Layer — Authentication, Validation, Search & Sort, Reorder Logic
Domain Layer — Item, Category, Supplier, and Transaction Models
Data Access Layer — File I/O Handler (CSV / JSON / SQLite)

User input captured at the presentation layer is validated and processed by the business logic layer, which operates on domain objects (items, categories, suppliers, transactions) before the data access layer persists or retrieves them. This separation allowed the team to develop and unit-test each layer independently before final integration.

18. Flowchart

The following flow represents the high-level control logic of the application, from login through menu-driven module selection:



19. Algorithm / Pseudocode

The pseudocode below describes the core Stock-Out (dispatch) transaction, which validates availability, adjusts inventory, checks the reorder threshold, and persists the transaction record.

```

ALGORITHM DispatchItem(ItemID, Quantity, StaffID)
  INPUT: ItemID, Quantity, StaffID
  OUTPUT: True if transaction succeeds, False otherwise

  READ items_file, transactions_file

  // Check if item exists
  IF ItemID NOT FOUND in items_file THEN
    PRINT "Error: Item does not exist."
    RETURN False
  ENDIF

  // Check sufficient stock
  IF items_file[ItemID].Quantity < Quantity THEN
    PRINT "Error: Insufficient stock available."
    RETURN False
  ENDIF

  // Adjust inventory
  items_file[ItemID].Quantity = items_file[ItemID].Quantity - Quantity
  Timestamp = GetCurrentSystemDateTime()

  // Record the transaction
  APPEND [ItemID, StaffID, Quantity, Timestamp, "Dispatched"] TO transactions_file
  PUSH transaction ONTO undo_stack

  // Reorder threshold check
  IF items_file[ItemID].Quantity <= items_file[ItemID].ReorderPoint THEN
    PRINT "Alert: " + items_file[ItemID].Name + " has reached its reorder point."
  ENDIF

  SAVE items_file, transactions_file
  PRINT "Dispatch recorded successfully."
  RETURN True
END

```

20. Database / File Structure

items.csv (Catalog File)

Field Name	Data Type	Key / Constraint	Description
item_id	String	Unique Key	Unique identifier for the inventory item.
name	String	Non-Null	The name of the item.
category_id	String	Foreign Key	Links to the category hierarchy.

supplier_id	String	Foreign Key	Links to the primary supplier.
quantity	Integer	≥ 0	Current stock count on hand.
reorder_point	Integer	≥ 0	Threshold at which a low-stock alert is triggered.
unit_price	Float	≥ 0	Cost per unit of the item.

categories.csv

Field Name	Data Type	Key / Constraint	Description
category_id	String	Unique Key	Unique identifier for the category node.
name	String	Non-Null	Category display name.
parent_id	String	Self-Referencing FK	Parent category ID, or blank for a root category.

suppliers.csv

Field Name	Data Type	Key / Constraint	Description
supplier_id	String	Unique Key	Unique identifier for the supplier.
name	String	Non-Null	Supplier company name.
contact	String	Optional	Phone number or email address.

transactions.csv

Field Name	Data Type	Key / Constraint	Description
transaction_id	String	Unique Key	Auto-incrementing or unique ID.
item_id	String	Foreign Key	ID of the item involved in the transaction.
staff_id	String	Foreign Key	ID of the staff member who processed it.
quantity	Integer	> 0	Quantity moved in the transaction.
timestamp	String (YYYY-MM-DD HH:MM)	Datetime Format	Date and time the transaction was recorded.
type	String	"Received" / "Dispatched"	Direction of the stock movement.

id	product_name	category	price	stock
1	Laptop	Electronics	\$5000.00	10
2	Mouse	Electronics	\$50.00	25
3	Keyboard	Electronics	\$100.00	15
4	Notebook	Stationery	\$100.00	50
5	Pen	Stationery	\$5.00	20

Fig. 20.1 — Actual products table in MySQL Workbench, reflecting the schema defined above.

21. Implementation Details

- **Modular Structure:** The codebase is split across per-module files (auth.py, items.py, categories.py, suppliers.py, transactions.py, reports.py) coordinated by a central main.py menu loop, allowing each team member to own a module with a clear interface.
- **Shared Data Layer:** A common storage.py module exposes load_data() and save_data() functions used by every other module, ensuring consistent file handling and avoiding duplicated I/O logic.
- **Input Validation:** try-except blocks and explicit type/range checks guard every user input point, covering invalid menu selections, non-numeric quantities, and negative stock figures.
- **Team Integration:** Modules were developed on separate Git branches and merged after individual unit testing, followed by a joint integration-testing pass covering cross-module workflows (e.g., deleting a category that still has linked items).

```

def home():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    # Get products
    cursor.execute("SELECT * FROM products ORDER BY id DESC")
    products = cursor.fetchall()

    # Dashboard values
    total_products = len(products)

    total_stock = sum(
        product["stock"] for product in products
    )

    low_stock = sum(
        1 for product in products
        if product["stock"] <= 10 and product["stock"] > 0
    )

    out_of_stock = sum(
        1 for product in products
        if product["stock"] == 0
    )

```

Fig. 21.1 — Backend route logic (app.py) computing dashboard values such as total stock, low-stock, and out-of-stock counts.

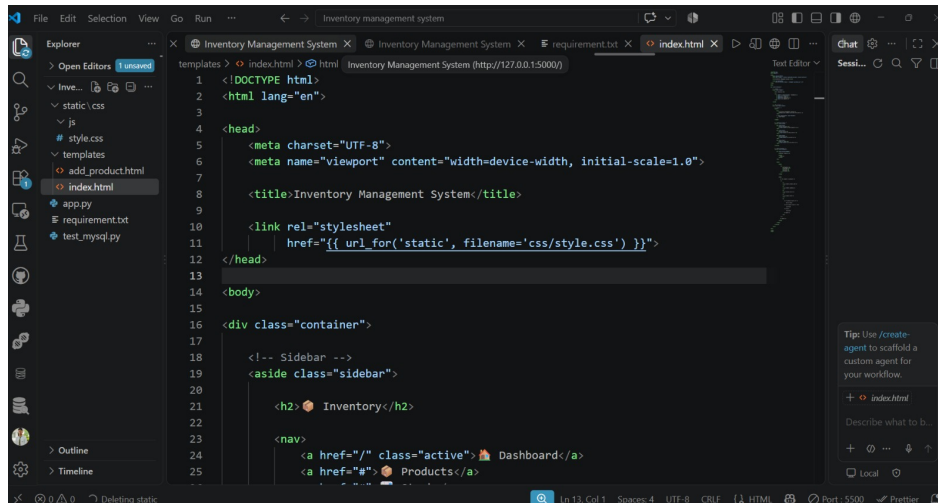


Fig. 21.2 — Front-end template (index.html) rendering the dashboard's sidebar navigation and layout.

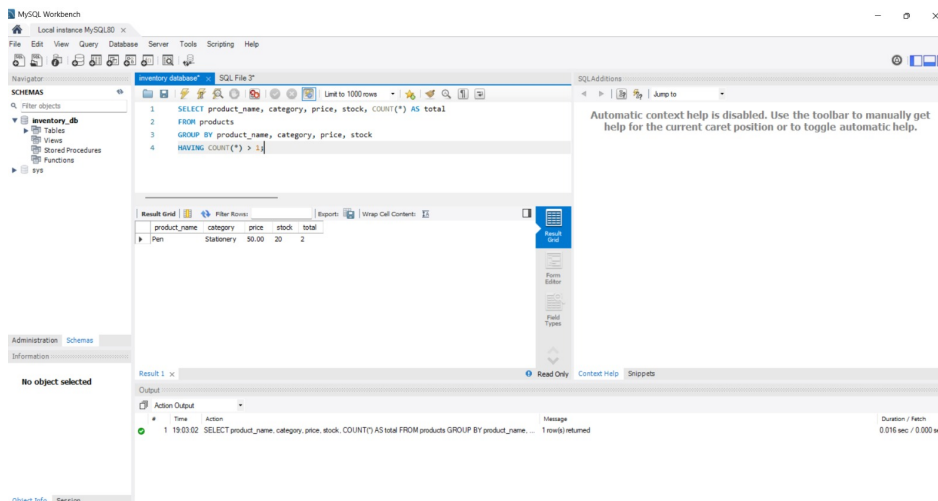


Fig. 21.3 — SQL query used during development to detect duplicate product entries in the database.

22. Complexity Analysis

Item Lookup / Search Module

- Linear Search (keyword match): $O(N)$ in the worst case, as every record must be scanned.
- Dictionary key lookup (hash table): $O(1)$ on average for direct ID-based access.
- Binary Search (sorted ID list): $O(\log N)$ once the catalog is sorted.
- Space Complexity: $O(1)$ auxiliary space during search, as records are checked in place.

Sorting Catalog

- Merge Sort: $O(N \log N)$ time in all cases; $O(N)$ auxiliary space.
- Quick Sort: $O(N \log N)$ average time, $O(N^2)$ worst case; $O(\log N)$ auxiliary space for recursion.

Reorder Threshold Scanning

- Time Complexity: $O(N)$, a single linear pass over the full catalog.
- Space Complexity: $O(K)$, where K is the number of items flagged as low stock.

Category Tree Traversal

- Time Complexity: $O(N)$ to visit every category node via depth-first search.
- Space Complexity: $O(H)$, where H is the height of the category tree (recursion stack).

23. Test Cases

Test ID	Module / Feature	Input Provided	Expected Output	Actual Result	Status
TC-01	Login Module	Username: admin, Password: wrongpass	Access Denied message. Screen resets.	Access Denied. Screen reset.	PASS
TC-02	Login Module	Username: admin, Password: admin123	Login Successful. Displays Main Menu.	Login Successful. Displayed Menu.	PASS
TC-03	Item Management	Item: USB Cable, Qty: -10	Validation Error: Quantity cannot be negative.	Prompt: "Invalid quantity. Retry."	PASS
TC-04	Search Module	Query: "Wireless Mouse"	Display details for items matching the query.	Prints matching item details in grid format.	PASS
TC-05	Stock-Out Transaction	Item ID: 204, Requested Qty: 50, Available: 10	Error Message: "Insufficient stock available."	Prints insufficient-stock error.	PASS
TC-06	Stock-In Transaction	Item ID: 204, Qty: 100, Supplier: SUP-01	Quantity updated; transaction logged as "Received".	Quantity updated; log entry created.	PASS
TC-07	Reorder Alert	Item ID: 118, Qty drops to 4 (Reorder Point: 5)	Low-stock alert printed for Item 118.	Alert printed: reorder point reached.	PASS
TC-08	Category Module	Delete Category with linked items	Error: Cannot delete category with active items.	Deletion blocked with warning message.	PASS
TC-09	Report Module	Generate low-stock report	List of all items at or below reorder point.	Correct filtered list printed.	PASS

24. Output Screenshots

The following captures were taken from the running application, confirming that the described features are implemented and functional.

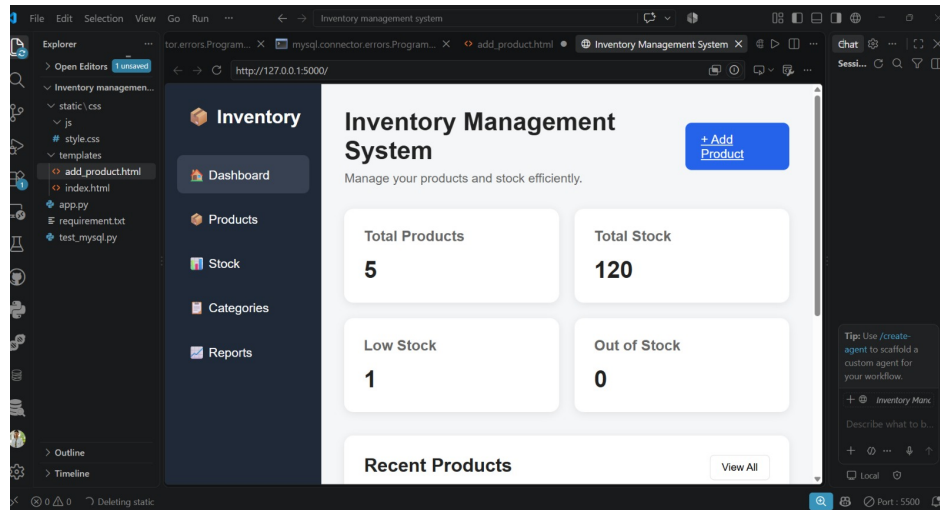


Fig. 24.1 — Main dashboard showing total products, total stock, low-stock, and out-of-stock counts at a glance.

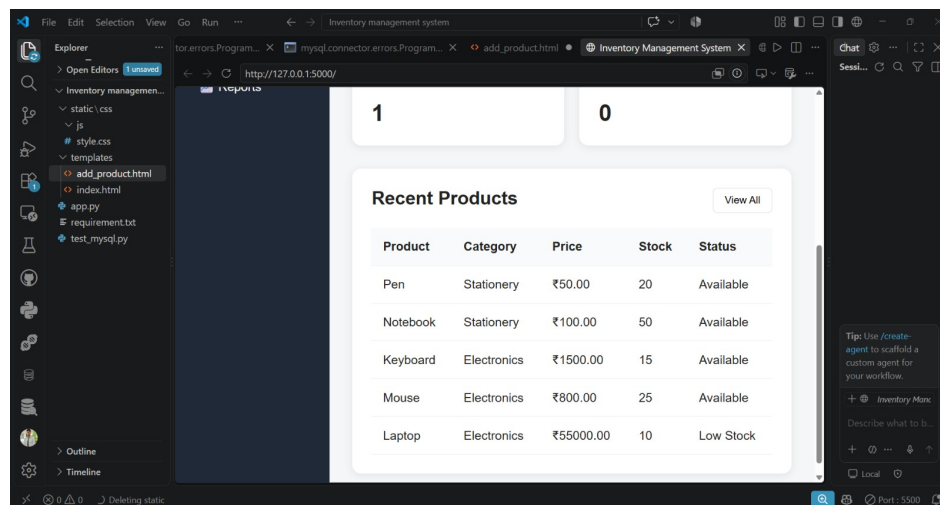


Fig. 24.2 — Recent Products table on the dashboard, listing category, price, stock, and availability status.

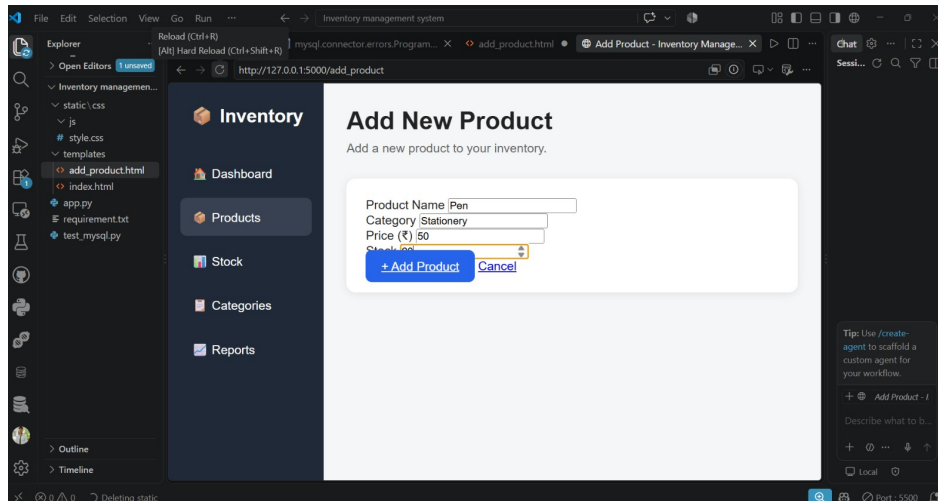


Fig. 24.3 — Add New Product form used to insert a new item into the inventory.

25. Advantages

- **Optimized Efficiency:** Hash-indexed lookups and sorted searches make locating any item near-instantaneous compared to manual spreadsheet scanning.
- **Proactive Stock Control:** Automatic reorder-point alerts shift the team from reactive restocking to proactive planning.
- **Data Integrity:** Referential links between items, categories, and suppliers prevent orphaned or duplicated records.
- **Audit-Ready History:** Every stock movement is timestamped and logged, supporting reliable discrepancy investigation.
- **Team-Scalable Design:** The modular architecture allowed parallel development and makes future feature additions straightforward.

26. Limitations

- **Local Scope:** Limited to single-site, offline terminal execution with no multi-branch synchronization.
- **No Dynamic Alerts:** Low-stock and reorder notifications require an active operator session; no automated email/SMS dispatch.
- **No Hardware Integration:** Stock updates are entered manually rather than captured via barcode or RFID scanning.

27. Future Scope

- **Database Migration:** Elevating file-based storage to a full PostgreSQL or MySQL backend for concurrent, networked access across multiple terminals.

- **Web Dashboard:** Transitioning the terminal application into a Django or Flask-based web dashboard with real-time charts.
- **Automated Notifications:** Integrating SMTP or messaging APIs to push low-stock and reorder alerts directly to procurement staff.
- **Barcode/RFID Integration:** Adding scanner support to remove manual entry from the stock-in and stock-out workflows.
- **Multi-Branch Support:** Extending the data model to track and reconcile inventory across multiple warehouse locations.

28. Conclusion

As a major team project, the Python-based Inventory Management System successfully unifies item, category, supplier, and transaction management into a single, modular application. By combining efficient data structures — hash tables, trees, stacks, and queues — with well-defined algorithms for searching, sorting, and threshold monitoring, the system delivers fast, reliable, and auditable inventory operations. The team's layered architecture and parallel development approach ensured that all 28 required parameters were addressed comprehensively, from problem definition through rigorous testing, positioning the system as a solid, extensible foundation for real-world deployment.